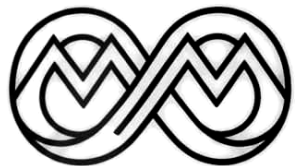


MLOPS MENA COMMUNITY PRESENTS

Data & Pipeline Versioning with DVC

Category 2 — Solving Git's Data Problem for Machine Learning Projects



MLOps MENA Community



What We'll Cover Today

1

The Problem

Why Git was never built to hold data, and what teams did about it before DVC.

2

Idea 5 — Versioning Large Datasets

How DVC keeps data out of Git, cache internals, and remote storage.

3

Idea 6 — Smart Pipelines with `dvc.yaml`

Building a DAG of stages, and how `dvc repro` reruns only what changed.

4

Idea 7 — Full-Stack Reproducibility

Linking a Git commit + a DVC data hash + an MLflow run into one traceable snapshot.

5

Quick Takeaways & Resources

The five-minute summary, plus where to go deeper.

Git Wasn't Built to Hold Data

1

Git diffs and merges text, line by line

It has no concept of what to do with a 2GB CSV or a folder of 50,000 images — it just stores the whole blob again.

2

GitHub hard-blocks files over 100MB

Pushes are rejected outright above 100MB, with warnings starting as early as 50MB.

3

History only ever grows

Every version of a large file you ever committed stays inside .git forever — the repo never shrinks back down, even if you delete the file today.

4

Cloning becomes painfully slow

A teammate cloning the repo has to download every historical version of every large file, just to get the latest code.

The Old, Painful Workarounds

1 Manual filename versioning
data_v1.csv, data_v2_final.csv, data_v2_final_REAL.csv — and nobody fully trusts any of them.

2 Sharing data via Drive links in Slack
Completely disconnected from the code and commit that actually used that data.

3 .gitignore-ing the data folder entirely
The repo can no longer reproduce anything on its own — you need someone's memory to know where the data lives.

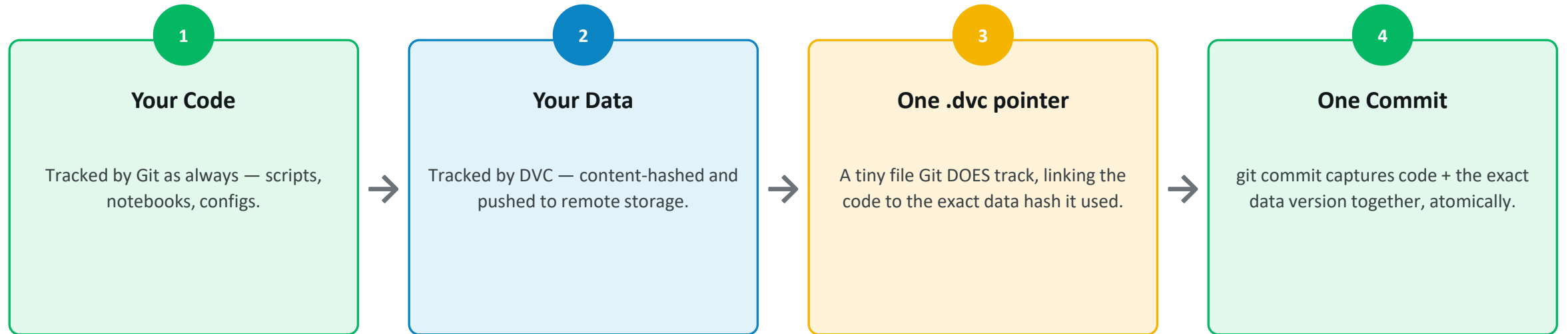
4 Git-LFS
Solves the size problem, but still charges for bandwidth/storage per repo and has zero awareness of pipelines — it doesn't know what depends on what.

THE FIX, IN ONE LINE

DVC keeps the heavy data on remote storage — S3, GCS, Azure — and leaves behind a tiny pointer file in Git that holds only a hash.

Git for code. DVC for data. Same commands, same habits, one workflow.

How Git and DVC Divide the Work



01

Versioning Large Datasets with DVC

Break the 100MB ceiling — without ever losing track of a single byte.



What Is DVC, Really?

1

An open-source CLI, built to sit next to Git

No servers, no databases to run — install with `pip install dvc` and you're versioning data in minutes.

2

The commands feel deliberately familiar

`add`, `commit` (via Git), `push`, `pull`, `checkout`, `status` — if you know Git, you already know 80% of DVC.

3

Storage-agnostic by design

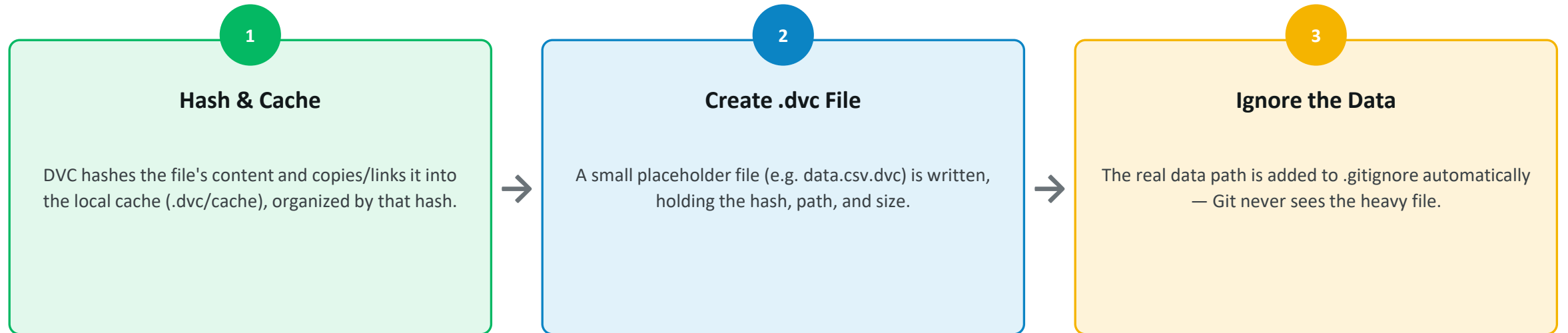
It doesn't lock you into one vendor: local disk, S3, GCS, Azure, SSH, or an on-prem NAS all work the same way.

4

Framework-agnostic too

DVC doesn't care whether you're training with scikit-learn, PyTorch, or a bash script — it only manages files and pipelines.

What Happens on `dvc add`



From here you `git add` and `git commit` the tiny `.dvc` file — never the dataset itself.

Anatomy of a .dvc File



yaml

```
# data.xml.dvc
outs:
  - md5: a304afb96060aad90176268345e10355
    path: data.xml
    size: 14445097
    desc: Cats and dogs dataset
    remote: myremote
```

md5 is the content hash used to detect any change, byte for byte — this whole file is a few hundred bytes, whether the dataset is 10MB or 100GB.

Where the Real Data Actually Lives: the Cache

- The DVC cache (.dvc/cache by default) is content-addressable storage — every file is stored once, named by its own hash.
- If two different datasets happen to share an identical file, DVC stores it only once and points both at it.
- The cache can be shared across multiple projects or even multiple machines, avoiding duplicate downloads entirely.
- This is the same mechanism Git itself uses internally for blobs — DVC just applies it to files too large for Git to handle well.

Cache path example

```
.dvc/cache/  
a3/  
04afb9...0355
```

The hash is split: first 2 characters become a subfolder, keeping any one directory small.

Linking Cache to Workspace: 4 Link Types

Link Type	Speed & Space	Trade-off
Reflink	As fast as a hard link, zero extra space	Safest option — copy-on-write, but only on Btrfs, XFS, APFS.
Hardlink	Fastest, zero extra space	Editing the file in place can corrupt the cache — DVC makes it read-only to protect you.
Symlink	Fast, zero extra space	Same corruption risk as hardlinks; also read-only by default, needs <code>dvc unprotect</code> to edit.
Copy	Slowest, doubles disk usage	Completely safe to edit — the fallback DVC uses when nothing faster is available.

Default behavior: DVC tries reflink first, then falls back to copy — configurable via `cache.type`.

Connecting to Remote Storage



bash

```
# Point DVC at an S3 bucket as the default remote
dvc remote add -d myremote s3://mybucket/dvcstore

# Push the actual data to S3, and the code + pointers to GitHub
dvc push
git push
```

Credentials go in `.dvc/config.local`, which is Git-ignored by default — secrets never get committed.

Supported Remote Storage Types

1

Amazon S3

Including any S3-compatible store,
e.g. MinIO.

2

Google Cloud Storage

GCS buckets, native auth support.

3

Azure Blob Storage

Native Azure credential support.

4

Google Drive

Good for small teams or quick
demos.

5

SSH / SFTP

Any server you can SSH into.

6

HDFS / WebHDFS

For on-prem Hadoop clusters.

7

HTTP / WebDAV

Any server exposing files that way.

8

Local / Network Folder

Zero setup — great for solo work.

Push & Pull: Syncing Data Like Git



bash

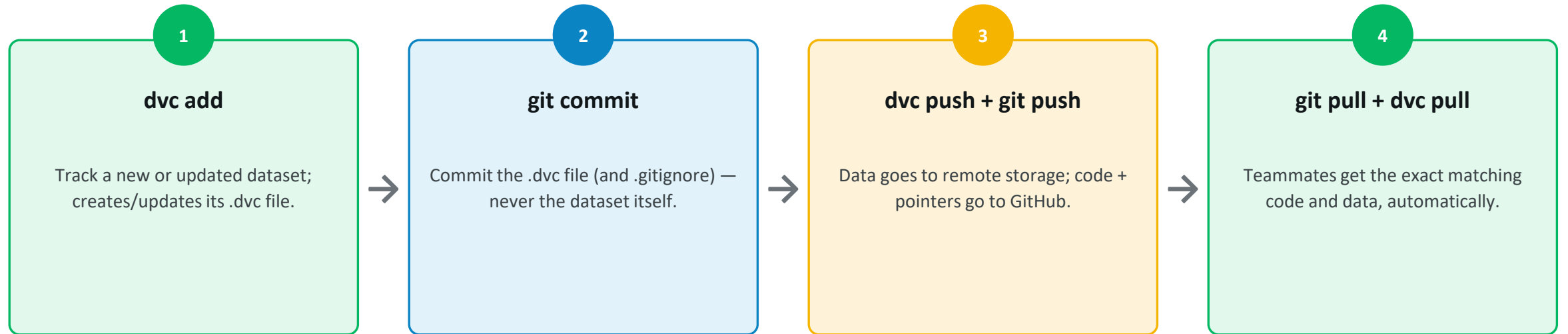
```
# Upload this workspace's data to the configured remote
dvc push
```

```
# Download the data referenced by the .dvc files you have
dvc pull
```

```
# Compare local cache/workspace against the remote
dvc status -c
```

Mental model: git push/pull move code + pointers; dvc push/pull move the actual bytes those pointers describe.

The Daily Workflow, End to End



Time Travel: Restoring an Old Data Version



bash

```
git checkout <old-commit>    # restores the old .dvc pointer file
dvc checkout                  # swaps workspace files to match that hash

# If that exact version isn't in your local cache yet:
dvc pull
```

Because .dvc files live in Git history, every past commit still points at the exact dataset hash used at that point in time.

Updating & Modifying Large Datasets

- When a tracked dataset changes, run `dvc add` again — DVC re-hashes it, updates the `.dvc` file, and caches the new version alongside the old one.
- Nothing is overwritten in the cache: both the old and new hash exist side by side, which is exactly what makes `git checkout` + `dvc checkout` able to time-travel.
- For datasets that only grow (new files appended, nothing removed), DVC only needs to hash and cache what's new — unchanged files are recognized instantly.
- `dvc status` tells you, before committing anything, exactly what changed since the last tracked version.

Rule of thumb

Treat `dvc add` + `git commit` as one atomic step — always pair them, the same way you'd never `git commit` code without testing it first.

Why This Matters

1

Consistency

Stable file paths in code forever — no more `data/20190922/labels_v7_final` to chase down.

2

Efficient storage

Content-addressed caching means identical files are never duplicated, even across many versions.

3

Cost control

Cheap object storage (S3/GCS) replaces paying for Git-hosting storage limits or LFS bandwidth.

4

Auditability

Every data change is a reviewable Git commit — or even a pull request — with a full, immutable history.

02

Smart Pipelines with dvc.yaml

Build a pipeline once — let dvc repro decide what actually needs to rerun.



A Pipeline Is a Dependency Graph

1

Stages are the building blocks

Each stage is one command, with declared dependencies and declared outputs.

2

DVC connects them automatically

One stage's output becomes the next stage's dependency — no manual wiring needed.

3

The result is a DAG

A Directed Acyclic Graph — `dvc dag` prints (or exports) it so the whole pipeline shape is visible at a glance.

4

Order in the file doesn't matter

DVC works out execution order from the dependency graph itself, not from the order stages are written in `dvc.yaml`.

Anatomy of dvc.yaml



yaml

```
stages:
  prepare:
    cmd: python src/prepare.py data/data.xml
    deps: [data/data.xml, src/prepare.py]
    params: [prepare.seed, prepare.split]
    outs: [data/prepared]

  featurize:
    cmd: python src/featurization.py data/prepared data/features
    deps: [data/prepared, src/featurization.py]
    params: [featurize.max_features, featurize.ngrams]
    outs: [data/features]

  train:
    cmd: python src/train.py data/features
    deps: [data/features, src/train.py]
    outs: [model.pkl]
```

prepare always runs before featurize, because data/prepared is prepare's output and featurize's dependency.

Three Kinds of Dependencies

Type	Declared As	What Invalidates the Stage
File / folder deps	deps:	Any byte-level change to that file's content hash.
Parameter deps	params:	Only the specific keys listed in params.yaml — unrelated keys in the same file don't count.
Outputs	outs:	Not a trigger themselves — they're cached and become the next stage's dependency.

Visualizing the DAG



bash

```
$ dvc dag
```

```
+-----+
| prepare |
+-----+
      *
+-----+
| featurize |
+-----+
      *
  +-----+
  | train |
  +-----+
      *
+-----+
| evaluate |
+-----+
```

One command, zero setup — the whole pipeline shape, straight from dvc.yaml.

dvc repro: Reproduce, Not Repeat



bash

```
$ dvc repro
'data/data.xml.dvc' didn't change, skipping
Stage 'prepare' didn't change, skipping
Running stage 'featurize':
> python src/featurization.py data/prepared data/features
Updating lock file 'dvc.lock'

Stage 'train' didn't change, skipping
```

Before running anything, DVC re-hashes every declared dependency and parameter and compares against the last successful run.

How the Hash Comparison Actually Works

1

Every dependency, parameter, and output gets hashed

Files by content hash, parameters by their exact value in params.yaml.

2

Hashes are compared against dvc.lock

The ledger written after the last successful dvc repro.

3

Unchanged stage → skipped entirely

If cached outputs already exist, DVC checks them out instantly instead of recomputing anything (the run cache).

4

Changed stage → reruns it and everything downstream

A change to featurize reruns featurize and train, but never touches prepare.

dvc.lock — The Reproducibility Ledger



yaml

```
# dvc.lock (excerpt)
stages:
  featurize:
    cmd: python src/featurization.py data/prepared data/features
    deps:
      - path: data/prepared
        md5: 2ecb...c9a1.dir
      - path: src/featurization.py
        md5: 9d17...44be
    params:
      params.yaml:
        featurize.max_features: 200
        featurize.ngrams: 2
    outs:
      - path: data/features
        md5: 71cb...0f88.dir
```

Auto-generated after every successful dvc repro, and committed to Git alongside dvc.yaml — comparing against it is literally how DVC decides what to skip.

Scaling Pipelines: matrix Stages



yaml

```
stages:
  train:
    matrix:
      model: [cnn, xgb]
      feature: [feature1, feature2, feature3]
    cmd: python train.py --model ${item.model} --feature ${item.feature}
    outs:
      - ${item.model}-${item.feature}.pkl
```

Expands into 6 real stages automatically — train@cnn-feature1 ... train@xgb-feature3 — each tracked and reproducible on its own.

Forcing, Skipping & CI Tricks

- `always_changed`: true on a stage forces it to rerun on every dvc repro — useful for stages pulling in live/external data.
- `dvc repro --dry --allow-missing` checks whether the pipeline is up to date without downloading or running anything — perfect for a fast CI gate.
- `dvc repro --pull --allow-missing` runs the pipeline while only pulling the data actually needed for stages that changed.
- `dvc stage add` can generate `dvc.yaml` entries from the command line, instead of hand-writing YAML.

CI-friendly check

```
dvc repro --dry
```

Tells you exactly which stages **WOULD** rerun — without touching a single file or running any code.

The Run Cache: Reuse Across Branches

- Every time a stage runs successfully, DVC stores its result keyed by the hash of its command + dependencies + outputs — the run cache.
- Switch to a different Git branch, and if a stage's inputs are identical to a run you already did anywhere else, DVC restores the result instantly instead of recomputing it.
- This works even without a matching dvc.lock — it's keyed purely on the stage's actual inputs, not on which commit you're in.
- The run cache can be shared across a whole team, the same way the data cache can — turning “someone already tried that combination” into a free result for everyone.

Think of it as

A cross-branch memory for every pipeline stage you've ever successfully run — not just the current checkout.

Why This Matters

1

No more “just rerun everything to be safe”

Only stages whose real inputs changed get recomputed — ever.

2

Expensive steps stay expensive only once

An hour-long feature engineering step doesn't rerun because someone tweaked the evaluation script.

3

Pipelines live in Git as plain YAML

Reviewable, diffable, and shareable through pull requests, exactly like code.

4

CI/CD becomes cheap to run on every push

--dry and --allow-missing let a pipeline check its own freshness before spending any compute.

03

End-to-End Reproducibility

Link a Git commit + a DVC data hash + an MLflow run into one traceable snapshot.



Three Sources of Truth, One Problem

1

Git knows the code

Exactly which commit hash produced the training script that ran.

2

DVC knows the data

Exactly which content hash, recorded in dvc.lock / .dvc files, was fed into that script.

3

MLflow knows the results

Exactly which metrics and parameters came out of a specific, named run.

4

The real question

If these three aren't linked together, how do you prove which code + which data produced which run, six months from now?

THE MISSING LINK

Record the Git commit and the DVC data hash inside the MLflow run itself — and a run stops being just numbers.

It becomes a fully addressable snapshot: code + data + results, in one place you can query later.

Logging Git + DVC Info Into an MLflow Run



python

```
import subprocess
import mlflow

def get_git_commit():
    return subprocess.check_output(
        ["git", "rev-parse", "HEAD"]
    ).decode().strip()

with mlflow.start_run():
    mlflow.set_tag("git_commit", get_git_commit())
    mlflow.log_artifact("dvc.lock")

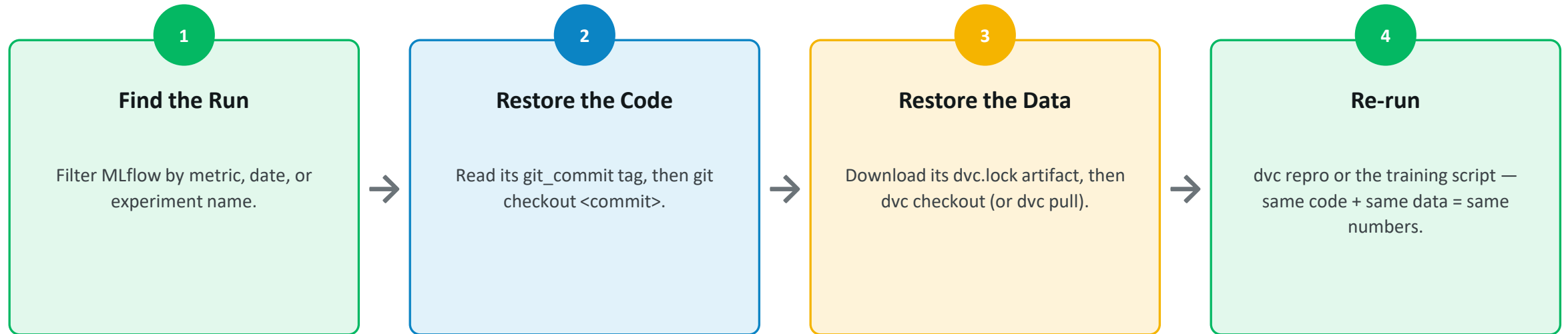
    mlflow.log_param("learning_rate", 0.001)
    ...
    mlflow.log_metric("f1_score", f1)
```

dvc.lock already contains the content hash of every dependency and output in the pipeline — logging it costs one line.

What Exactly to Log, and Why

- 1 git_commit tag**
Pins the exact version of every script, notebook, and config used in this run.
- 2 dvc.lock as an artifact**
Carries the content hash of every pipeline input and output, with zero extra bookkeeping.
- 3 The active pipeline stage name**
Useful when one MLflow experiment covers several dvc.yaml stages (e.g. different train@ variants).
- 4 Any manually-touched data path**
For one-off scripts outside dvc.yaml, log the specific .dvc file's hash as an MLflow param.

Reconstructing a Run From Months Ago



The Reproducibility Triangle

Without the Link

- “Which CSV did you use for this?” becomes a guessing game
- Reproducing an old result means digging through Slack history
- A metric regression can't be traced to its actual cause
- Audits and compliance checks rely on memory, not evidence

With Git + DVC + MLflow

- Every MLflow run carries its own code and data fingerprint
- Any past result is one checkout + one repro away
- Regressions are diffed: two commits, two data hashes
- Full lineage is provable, not just remembered

What Breaks This Chain

1

Forgetting to commit `dvc.lock`

Without it in Git, the exact data hashes for that run are lost forever — log it as an MLflow artifact as a safety net.

2

The remote storage gets cleaned up

A `.dvc` pointer with no matching object in the remote can't be pulled back — treat remotes like backups, not scratch space.

3

Local-only cache, never pushed

`dvc add` without `dvc push` means the data version only exists on one laptop.

4

Logging metrics without tagging the commit

The single most common gap — a great MLflow run with no way to know what code produced it.

Why This Matters

1

Debugging regressions

“The metric dropped last month” becomes “diff these two commits and these two data hashes,” not a guessing game.

2

Compliance & audits

Prove precisely what data and code produced a deployed model, on demand.

3

Team trust

Anyone can reproduce anyone else's result, without ever asking “which version of the CSV did you use?”

4

Future-you says thank you

The result you need to defend “in six months” is exactly the one this pattern protects.

The Five-Minute Summary

1

DVC = Git for data

Heavy files live in remote storage; Git only keeps a small hash-based pointer.

2

.dvc files solve the 100MB problem

The actual data never touches Git — only its content hash does.

3

dvc.yaml + dvc repro = a smart pipeline

Only stages whose real inputs changed get recomputed — verified against dvc.lock.

4

dvc.lock is the bridge to MLflow

The exact ledger that makes smart reruns possible, and the artifact that makes runs reproducible.

5

Git + DVC + MLflow together

Full lineage — code, data, and results — all traceable from a single MLflow run, months later.



Thank You!

Questions, discussion, and more MLOps content — join us on our channels.

- doc.dvc.org/start — Get Started guide
- doc.dvc.org/user-guide/project-structure/dvcyaml-files — full dvc.yaml reference
- doc.dvc.org/user-guide/data-management/remote-storage — all supported remote types
- github.com/iterative/dvc — source & issues

